DOYENSEC

Security Auditing Report

ChatGPT for Windows
Q3 2024

Prepared for: OpenAI, Inc.
Prepared by: Mohamed Ouad
Date: 10/10/2024

Table of Contents

Revision History

Version	Date	Description	Author
1	10/08/2024	First release of the final report	Mohamed Ouad
2	10/10/2024	Peer Review	Luca Carettoni

Contacts

Company	Name	Email
OpenAI, Inc	Shino Jomoto	shino@openai.com
Doyensec, LLC	Luca Carettoni	luca@doyensec.com
Doyensec, LLC	John Villamil	john@doyensec.com

Executive Summary

Overview

OpenAI, Inc. engaged Doyensec to perform a security assessment of the ChatGPT Windows Desktop application. The project commenced on 09/30/2024 and ended on 10/04/2024 requiring one (1) security researcher for a total of five (5) person/days. The project resulted in five (5) findings with no critical or high severity issues.

The project consisted of a manual Desktop application security assessment.

Testing was conducted remotely from Doyensec's EMEA and US offices.

Scope

Through meetings with OpenAI, Inc. the scope of the project was clearly defined. The agreed-upon assets are listed below:

- ChatGPT Windows Desktop

The testing took place in a development environment using the latest version of the software at the time of testing. In detail, this activity was performed on the following releases:

- ChatGPT Windows Desktop - **v. 2.0.0**
  - sidetron.zip - (sha256: `5f2ca3cae32bc69639d239425fa8f6d4854da97fd44bfe249da7263e5b5de59a`)

Scoping Restrictions

During the engagement, Doyensec did not encounter any major difficulties testing the functionalities of the application.

Findings Summary

Doyensec researchers discovered and reported five vulnerabilities in the ChatGPT Windows Desktop. All of the issues were departures from best practices and they are considered low-severity flaws.

It is important to reiterate that this report represents a snapshot of the environment's security posture at a point in time.

The findings included an insufficient deletion of application data, insecure logging exposing sensitive information such as conversation attachments or chat sharing urls and an overprivileged permissions allowance vulnerability.

Overall, the security posture of the Internet-facing APIs was found to be above industry best practices.

At the design level, Doyensec found the system to be well architected with the exclusion of the following aspects:

- ChatGPT application login content from a remote source
- Browser permissions granted without a read need

Recommendations

The following recommendations are proposed based on studying ChatGPT's security posture and the vulnerabilities discovered during this engagement.

Short-term improvements

- Work on mitigating the discovered vulnerabilities. You can use **Appendix B** - Remediation Checklist to make sure that you have covered all areas.

Methodology

Overview

Doyensec treats each engagement as a fluid entity. We use a standard base of tools and techniques from which we built our own unique methodology. Our 30 years of information security experience has taught us that mixing offensive and defensive philosophies is the key to standing against threats. Thus we recommend a *whitebox* approach combining dynamic fault injection with an in-depth study of the source code to maximize the ROI on bug hunting.

During this assessment, we have employed standard testing methodologies (e.g., OWASP Testing guide recommendations), as well as custom checklists, to ensure full coverage of both code and vulnerability classes.

Setup Phase

OpenAI, Inc. provided access to the online environment, source code repositories and the binaries for the Desktop.

The following accounts have been used during the testing activity:

- mohamed+penstest@doyensec.com
- testing1@doyensec.com

The access to the app release has been provided via Microsoft Store (ms-windows-store://pdp/?productid=9NT1R1C2HH7J&ocid=ggWImO1CEmiYtBgq3WhJEkOYKgJdDDwgGQ)

Tooling

When performing assessments, we combine manual security testing with state-of-the-art tools in order to improve efficiency and efficacy of our effort.

During this engagement, we used the following tools:

- Burp Suite
- VS code
- ElectroNG
- Asar
- Sysinternals Suite
- DLLHijackTest
- Curl, netcat and other Linux utilities

Electron Apps Testing

Doyensec was the first security company to publish a comprehensive security overview of the Electron framework during BlackHat USA 2017. Since then, we have reported dozens of vulnerabilities in the framework itself and many popular Electron-based applications.

Thanks to our research efforts, we have extensive experience in analyzing desktop runtime environments based on web technologies. During our testing effort, we will review security mechanisms that ensure isolation between sites, facilitate web security protections and prevent untrusted remote content to compromise the security of the host. We write custom tools to map out control flow and study an application's behavior and internals. Mapping out the attack surface, whether local or remote, is paramount to a successful engagement. Doyensec also studies the application's ecosystem, looking for potential pitfalls and common misconceptions.

Static analysis and instrumentation are important parts of the testing process we use to test an application's response to untrusted data. Doyensec combines manual security testing with a mature Electron.js security testing tool (https://github.com/doyensec/ electronegativity) which will be customized to meet the needs of the target.

We take apart the application looking for privacy leaks and secrets. Storage, transmission, and protection of user information is critical.

Source Code Auditing

Source code reviews are critical to understand the true state of our clients' applications. Removing the layers of abstraction and focusing on the raw application code allows us to obtain an unobstructed view of vulnerabilities, which could otherwise go undetected. Unlike black-box testing, which can be impeded by things like network equipment, security devices and services, rate limits, hard to find routes or inputs, code obfuscation and/or minimization and the fear of creating downtime, source code reviews reveal the true software quality. This gives our clients the confidence to deploy their software anywhere, knowing we've found the hidden vulnerabilities.

We pride ourselves on hiring engineers who not only break applications, but have experience building them as well. This differentiates Doyensec from many other consulting firms, as it allows us to immerse ourselves in the applications we test and create a more comprehensive threat model, ultimately revealing more impactful issues and in greater numbers.

Our methodology consists of several stages outlined in the table below. During this process, we typically begin by thoroughly reviewing the code to understand the composition of the application, its uses, data flows and its authentication and authorization structures. This provides the context needed to evaluate any potential bugs we might encounter in later steps.

Next, we use our custom threat model to trace inputs through the code and look for problem areas. These could be anything from typical bugs like injection style vulnerabilities and authorization bypasses, or subtle business-logic flaws and unsafe function usages, which are not as easily detected via automation. We also examine the frameworks used within the application to ensure they are configured as securely as possible for the given context.

Only once we really understand the application, will we deploy our custom scripts, created by our team over many years of experience. These parse the code and identify hotspots, where we have seen vulnerabilities manifested in the past. Our scripts typically identify coding errors and misconceptions about functionality during development and are the product of bespoke security research by our team.

Finally, we leverage a curated set of customized open-source tools to complete our analysis. We start with those specifically designed for the application's languages and components, eventually moving into more generalized tools. After assessing the application with these tools, we manually validate any findings they report and filter out the noise, delivering only actionable results to our clients.

Project Findings

The table below lists the findings with their associated ID and severity. The severity ranking and vulnerability classes are defined in **Appendix A** at the end of this document. The vulnerability class column groups the entry into a common category, while the status column refers to whether the finding has been fixed at the time of writing.

This table is organized by time of discovery. The issues at the top were found first, while those at the bottom were found last. Presenting the table in this fashion has a number of benefits. It inherently shows the path our auditing took through the target and may also reveal how easy or difficult it was to discover certain findings. As a security engagement progresses, the researchers will gain a deeper understanding of a target which is also shown in this table.

Findings Recap Table

ID	Title	Vulnerability Class	Severity	Status
CHA-Q324-1	Windows DLL Search Order Hijacking	Injection Flaws (SQL, XML, Command, Path, etc)	Informational	Open
CHA-Q324-2	Insecure Logging Exposes Sensitive Data	Information Exposure	Low	Open
CHA-Q324-3	Insufficient Deletion of Application Data	Information Exposure	Low	Open
CHA-Q324-4	Application Loading Remote Content	Insecure Design	Informational	Open
CHA-Q324-5	Overprivileged Permission Allowance	Insecure Design	Low	Open